

RACCOLTA DOMANDE DI RAGIONAMENTO

(2018-2019)

Sicuramente ci sono errori, per proporre correzioni scrivetemi su telegram @zetaexe

Spiegare perché il datapath multiciclo è più veloce di quello singolo ciclo

Perché il datapath a ciclo singolo richiede un tempo fissato di 1 ciclo di clock (comunque più lento del datapath multi-ciclo) di durata uguale alla durata di esecuzione dell'istruzione che richiede più tempo (lw), rendendo meno efficiente l'esecuzione delle istruzioni che ne richiedono di meno.

Meno efficiente anche perché nel datapath multi-ciclo invece, le istruzioni vengono "spezzate" in più cicli di clock (impiegando comunque meno del ciclo singolo), inoltre il datapath multi-ciclo permette di riutilizzare alcuni elementi per operazioni future ed è provvisto di registri parziali in cui salva i dati che potrebbe riutilizzare nel ciclo successivo. Riguardo al riutilizzo di alcuni elementi bisogna anche notare che nel datapath a ciclo singolo per riutilizzare un elemento è necessario duplicarlo, aumentando i costi.

Spiegare cosa bisogna implementare per salvare il dato presente in ALUout per riutilizzarlo in prossimo ciclo di clock

Innanzitutto parto col salvare il dato output di ALUout in un registro aggiuntivo.

Estendo poi il multiplexer gestito da lorD (quello presente subito dopo il PC), aggiungendo come scelta possibile anche il suddetto registro, così da poter utilizzare il dato nel ciclo successivo, qualora servisse.

Pertanto alla fine il multiplexer da 2 ingressi passerà ad averne 3, e di conseguenza aggiungerò un segnale di selezione, modificando anche lorD.

Se invece si intende usare l'output di ALUout come vero e proprio operando, aggiungerò comunque un registro in cui salvare il dato ma invece di estendere il multiplexer di lorD farò sì che il suddetto registro sia uno dei possibili ingressi del multiplexer gestito da ALUSrcA, aggiungendo anche qui il terzo ingresso.

Modifica il datapath: Implementare l'istruzione jal senza ALU

Modifico il mux controllato da RegDst (aggiungendo l'ingresso 10 fisso a \$ra)

Collego PC al mux controllato da MemToReg (aggiungendo l'ingresso 10)

La FSM sarà la seguente:

- RegDst = 10
- MemToReg = 10
- PCSrc = 10
- PCWrite
- RegWrite

Modifica il datapath: Implementare l'istruzione jr senza ALU

Non serve modificare il datapath, è solo necessario impostare la SFM nel seguente modo:

- ALUSrcA = 1 (il registro con l'indirizzo a cui dobbiamo saltare)
- ALUSrcB = 0 (poiché vale 0 nella jr)
- ALUOp = 00 (somma)
- PCSrc = 00 (l'output dell'ALU)
- PCWrite

Dato il seguente datapath (a singolo ciclo), con i bit dell'Instruction Register da 0-15 tutti azzerati, esegui una jump register

Per eseguire una jr bisogna aggiungere un multiplexer affinché venga scelto il valore corretto per il PC.

Per fare ciò le linee di controllo della Control Unit devono assumere i seguenti valori:

- Branch = 0
- MemRead = 0
- MemWrite = 0
- RegWrite = 0

Tutti gli altri (RegDst, MemReg, ALUOp, ALUSrcA/B non vengono utilizzati)

Si spieghi la differenza, nell'architettura MIPS32, tra i salti incondizionati "j" e quelli "jr"

- *j: Jump*, è un salto incondizionato all'istruzione che si trova all'indirizzo di destinazione
es. j destinazione
Struttura: opcode (6 bit) = 3 | destinazione (26 bit) = x
Il PC assume come valore l'indirizzo destinazione. Essendo però quest'ultimo di 26 bit, bisogna fare uno shift left di due posizioni (aggiungendo 2 zeri a destra) e concatenare a sinistra con i 4 bit più significativi del PC (quelli più a sinistra)
- *jr: Jump Register*, salto incondizionato all'istruzione il cui indirizzo si trova nel registro rs
es. jr rs
Struttura: opcode (6 bit) = 0 | rs (5 bit) = x
rt + rd + shamt (15 bit) = 0 | funct (6 bit) = 8

Vantaggi e svantaggi di avere più di 2 registri per il kernel

Svantaggi:

- i registri per l'utente avranno più accessi in memoria da fare, e ciò renderebbe la macchina più lenta

Vantaggi:

- Il kernel deve salvare e fare il restore di meno registri, pertanto il gestore delle eccezioni sarà più veloce

Vantaggi e svantaggi di avere uno stack separato dedicato al kernel

Vantaggi:

- Con uno stack separato dedicato al kernel si ha un'area di memoria riservata, ergo l'utente non può accedervi e rovinare in qualche modo i dati.
- Quando il kernel salva qualcosa nel suo stack esso non verrà modificato.

Svantaggi:

- Non avendo un'area di memoria riservata, non si può sapere se il kernel potrà effettivamente operare
- Si potrebbero creare errori non risolvibili tipo stack overflow
- Se lo stack è già pieno e chiamo il gestore delle eccezioni per uno stack overflow, esso proverà a scrivere sullo stack (per salvare i registri) creando un altro stack overflow

Spiegare perché usare una cache per i dati ed una per il text sullo stesso livello

Si fa ciò perché in questo modo è possibile applicare due politiche di locazione diverse.

SPIEGAZIONE: Il text è contiguo, ossia i dati presi da esso (le istruzioni) sono in gran parte sequenziali, quindi posso sfruttare la località spaziale.

Mentre per il data segment non posso sapere quanto sia sequenziale, pertanto è meglio utilizzare una cache che sfrutta la località temporale piuttosto che quella spaziale.

Vantaggi e svantaggi di avere due cache sullo stesso livello

Avendo due cache ne uso una per il text segment e una per il data segment, perciò ho come vantaggio la possibilità di sfruttare la località spaziale per il text segment (che so essere "contiguo", sequenziale)

Mentre per quanto riguarda il data segment non posso sapere quanto siano sequenziali i dati pertanto se sfruttare la località temporale è la scelta migliore.

La cache dedicata al text sarà con pochi blocchi di grosse dimensioni per sfruttare al meglio la località spaziale (un unico tempo di caricamento in cache per una grande quantità di dati)

La cache dedicata al data sarà con tanti blocchi seppur di dimensioni minori, così da avere più "parti" di dati a disposizione in cache, per sfruttare al meglio la località temporale.

Si discuta la correttezza dell'affermazione "la gestione delle periferiche a controllo di programma consente di gestire le periferiche più critiche in termini di latenza e capacità di trasferimento dati (banda passante) di quelle gestibili con la tecnica interruzione di programma".

L'affermazione è corretta: nella gestione delle periferiche a controllo di programma la latenza è minima in quanto la CPU si accorge subito se la periferica è in stato di ready, e la banda passante è molto alta in quanto la gestione della periferica richiede solo poche istruzioni.

Nel caso del sistema di interruzione è la periferica a contattare la CPU e a farla "bloccare" per eseguire il suo protocollo. Di conseguenza, per l'interruzione di programma avremo latenza massima perché ci sono più operazioni da eseguire per contattare la CPU, e la banda passante è minima perché il trasferimento di ciascun dato impiega più tempo.

Calcola la banda passante di una periferica con controllo di programma dati solo la frequenza del clock e il tempo di accesso a memoria.

- 1) Uso la frequenza per calcolare il periodo di un ciclo
- 2) Ipotesizzo il numero di cicli necessari per ogni istruzione (essendo libero di fare delle assunzioni, per semplificare il problema decido che ogni tipo di istruzione impiega 5 cicli)
- 3) Calcolo quanti millisecondi sono necessari per l'esecuzione di ogni istruzione
- 4) Descritto le operazioni necessarie per implementare il controllo di programma, (senza scriverle in assembly, ma descrivendo il loro utilizzo)
- 5) Una volta contate quante istruzioni assembly servono per implementare il controllo di programma, ho moltiplicato per la durata in millisecondi di ogni istruzione, ottenendo il numero di millisecondi necessari per trasferire una word dal registro dati della periferica in modo da "metterlo in salvo" prima che venga sovrascritto (cioè la latenza della periferica)
- 6) sapendo ora la latenza e ovviamente anche la quantità di dati trasmessi ad ogni trasferimento (una word, quindi 32 bit) ho costruito la frazione della banda passante in "bit al millisecondo"

ora basta una conversazione per ottenere la banda passante in Mbit/s

Calcolare massima banda passante data la frequenza del processore e la velocità degli accessi in memoria

- CPU a 100 Mhz

- Controllo di programma

- Lettura/scrittura 7ns

Ricordo che: $Frequenza = n \text{ cicli} / tempo$

(e la frequenza in Hz indica il numero di cicli al secondo)

1) Calcolo quanti cicli vengono eseguiti nel fare una lettura o scrittura da 7ns:

$n \text{ cicli} = \text{frequenza} * \text{tempo}$

$$= (100 * 10^6 \text{ cicli/secondo}) * (7 * 10^{-9} \text{ secondi}) = 0.7 \text{ cicli}$$

2) Dato che lettura e scrittura impiegano lo stesso tempo avrò un totale di $0.7 * 2 = 1.4$ cicli

3) Calcolo il tempo impiegato per compiere 1.4 cicli :

$$\text{Tempo} = n \text{ cicli} / \text{frequenza} = 1.4 \text{ cicli} / (100 * 10^6 \text{ cicli/secondo}) = 1.4 * 10^{-8} \text{ secondi}$$

Un registro da 32 bit viene trasferito in questo lasso di tempo.

4) Calcolo la banda massima:

banda massima = (quante volte avviene il trasferimento precedente in 1 sec) * 4 byte (32 bit)

$$= [1 / (1.4 * 10^{-8}) \text{ secondi}] * 4 \text{ byte} = 285714285.7 \text{ byte} = 285 \text{ MB / secondo}$$

Massima banda passante di periferiche gestite a controllo di programma con:

- CPU a 600mhz (quindi $6 * 10^6$)

- Mappate in memoria

- Lettura 5ns scrittura 15ns

1) Calcolo il numero di cicli per lettura/scrittura

$$\text{Cicli lettura} = \text{frequenza} * \text{tempo lettura}$$

$$(6 * 10^6) * (5 * 10^{-9}) = 6 * 5 * 10^{-1} = (6 * 5) / 10 = 30/10 = 3 \rightarrow 3 \text{ cicli lettura}$$

$$\text{Cicli scrittura} = \text{frequenza} * \text{tempo scrittura}$$

$$(6 * 10^6) * (15 * 10^{-9}) = 6 * 15 * 10^{-1} = (6 * 15) / 10 = 90/10 = 9 \rightarrow 9 \text{ cicli scrittura}$$

2) Calcolo il numero dei cicli totali

$$\text{Numero cicli totali} = \text{scrittura} + \text{lettura} = 12 \text{ cicli}$$

$$\text{Tempo cicli totali} = \text{numero cicli} / \text{frequenza}$$

$$12 / (6 * 10^6) = 2 / 10^6 = 2 * 10^{-6}$$

Banda passante = $(1 / \text{tempo cicli totali}) * 4 \text{ byte}$

$$[1 / (2 * 10^{-6})] * 4 \text{ byte} = (1/2 * 1/10^{-6}) * 4 = (1/2 * 10^6) * 4 = 2 * 10^6 = 200 * 10^6 = 200 \text{ Mb/s}$$

Una CPU MIPS esegue 1 istruzione ogni ciclo di clock

- Il clock della CPU è di 500 MHz // $500 * 10^6$ hertz oppure $5 * 10^8$ hertz

- Il device emette al massimo 1 KB/s (1000 byte/s)

- Impostare i registri del DMA occupa 10 cicli di clock

- Prologo/epilogo del controllo di programma trascurabili

Si vuole trasmettere un testo di 2Mb // $2 * 10^6$ byte

Qual è il rapporto dei tempi di impiego della CPU nel caso di DMA e di I/O a controllo di programma?

Controllo di programma:

Tempo trasferimento = byte da trasferire / Banda

$= (2 * 10^6 \text{ byte/s}) / 1000 \text{ byte} = 2 * 10^3 \text{ secondi}$

DMA: la CPU è usata solo per i 10 cicli di clock dell'impostazione dei registri

Tempo per ciclo = 1 / Frequenza

$= 1 / (5 * 10^8)$

*Tempo totale del trasferimento = (cicli di clock per impostare i registri del DMA) * tempo per ciclo*

$10 * [1 / (5 * 10^8)] = 1 / (5 * 10^7) = 1/5 * 1/10^7 = 0,2 * 10^{-7} = (2 * 10^{-1}) * 10^{-7} = 2 * 10^{-8}$

Poi faccio un rapporto dei tempi del controllo di programma con quelli del DMA, ossia:

$(2 * 10^3) / (2 * 10^{-8}) = 10^3 * 10^8 = 10^{11}$

Si ha un programma di 1024 byte a un indirizzo 0x1010 0000

Il programma ha all'interno una serie di cicli che sistemano un array di 1024 byte indirizzato all'indirizzo 0x1010 0000 in due array distinti ai rispettivi indirizzi:

0x1020 0000 e 0x1030 3000

La cache ha una dimensione di 4 KB. Quanto devono essere grandi e sistemati i blocchi di questa per fare meno miss possibili?

Minimizzare le miss = avere blocchi atomici, ossia con una dimensione maggiore o uguale a quella dell'array.

Pertanto se scelgo 4 blocchi da 1024 byte (4 Kb), di cui uno per il programma e i restanti tre per l'array originale e le due "copie", così ciascun blocco contiene interamente un array.

Così le miss sarebbero state:

1) per caricare il programma, che era ad un indirizzo lontano (non ricordo dove)

2) quella per caricare l'array pieno

3) quella per caricare gli array 'cloni'.

Poi mi sembra che il programma faceva non so quanti loop ma a te non fregava niente perché avevi già tutto caricato in cache

Per diminuire le miss posso aumentare la dimensione del primo livello, aumentando così il numero di hit.

La pseudoistruzione `li, $rd, imm` (carica in `$rd` una costante) ha due traduzioni possibili in MIPS32.

La prima è l'istruzione `ori, $rd, $0, imm` (`rd = $0 OR imm`) usata quando `imm` è rappresentabile in 16 bit.

La seconda è la seconda Traduzione Che Non Ricordo

Quale traduzione è usata se `imm = -0x0100` ?

Vedo quanto vale l'immediato:

- se $\geq 2^{16} \rightarrow$ seconda istruzione
- se $\geq 2^{16} \rightarrow$ prima istruzione (`$rd`)

Si guarda se la rappresentazione in binario CA2 di `imm` è possibile in 16 bit.

Altrimenti, essendo che `imm` è negativo, si guarda se $|\text{imm}| \leq 2^{15}$

`-0x0100` è rappresentabile in 16 bit?

Per capirlo, lo trasformo da esadecimale in binario CA2

Considero il modulo di `-0x0100`, dunque `0x0100 = 0000000100000000`

Eseguo il complemento A 1 e sommo 1 e in questo modo ottengo la rappresentazione binaria di `-0x0100`

`1111111011111111 + 0000000000000001 = 1111111100000000 (= - 0x0100)`

Determino ora il range secondo la formula: $-(2^{n-1})$; $+(2^{n-1} - 1)$

In questo caso $n = 16$

Quindi $-(2^{15})$; $+(2^{15} - 1)$ e di conseguenza:

`1000000000000000` ; `0111111111111111`

Perciò il nostro valore è compreso nell'intervallo considerato.

In base al testo, essendo rappresentabile in 16 bit, viene scelta la prima risposta.

Estendere la codifica ASCII mettendo a ogni carattere una sua variante con un accento e Scrivere un modo per passare da una variante all'altra in 3 istruzioni assembly.

In particolare con 3 istruzioni assembly eseguire:

- Aggiungere al simbolo l'accento
- Rimuovere dal simbolo l'accento
- Branch se è accentato

Estendo la codifica ASCII a 7 bit considerando ora 8 bit. In questo modo tratto i caratteri come se fossero numeri in complemento a 2, ovvero usando il nuovo bit (l'ottavo) come segno. Nel caso di un numero negativo significa che si ha un carattere accentato, nel caso di uno positivo significa che non c'è l'accento. In questo modo si mantiene anche la compatibilità con il vecchio ASCII.

Aggiunta dell'accento al simbolo:

```
la $t0, address
lb $t1, 0($t0)
sub $t1, $zero, $t1
```

Branch se con accento:

```
la $t0, address
lb $t1, 0($t0)
bltz $t1, label
```

Fornire il codice binario della seguente istruzione:

seq rdest, rsrc1, rsrc2

e se non possibile, di scrivere un breve codice che facesse la stessa cosa, e poi di trasformare quel codice in binario

Imposta ad 1 il registro rdest se il contenuto del registro rsrc1 è uguale a quello del registro rsrc2, altrimenti imposta 0.

Essendo una pseudoistruzione la converto in più istruzioni in modo tale da poterla convertire in binario:

Seguo le convenzioni MIPS:

$\$rsrc1 = \$t0 \mid \$rsrc2 = \$t1 \mid \$rdest = \$s0$

Dunque:

```
    beq $t0,$t1, label
```

```
    ori $s0, $zero, 0
```

```
    beq $zero, $zero, label2
```

```
label1:
```

```
    ori $s0,$zero, 1
```

```
label2:
```

```
    #end
```

// e poi lo trasformo in binario.